Scientific Computing

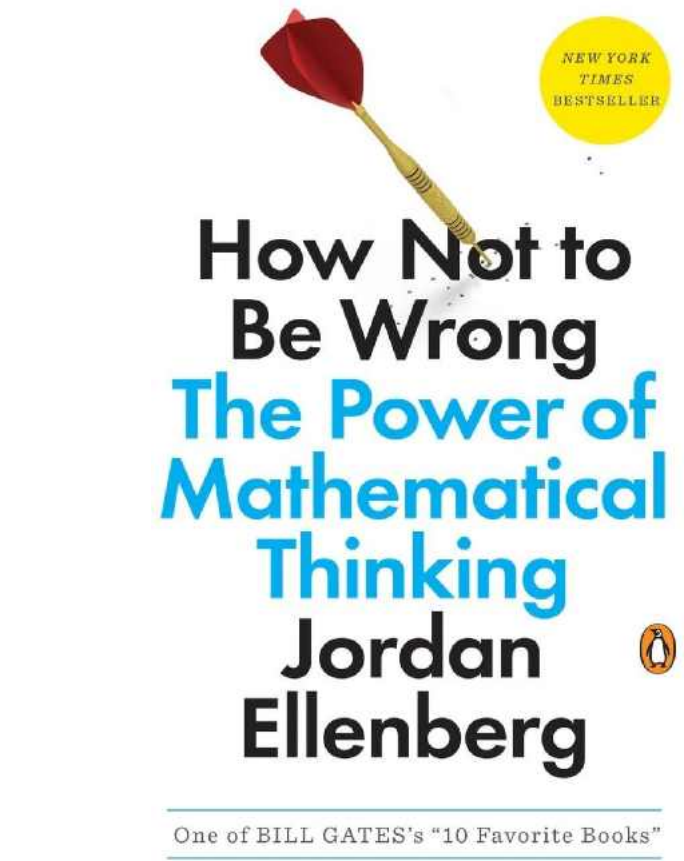
L02 Numbers

Thorsten Koch

Applied Algorithmic Intelligence Methods department at ZIB

Chair of *Software and Algorithms for Discrete Optimization* at TU Berlin

Book recommendation of the week



How Not to Be Wrong explains the mathematics behind some of simplest day-to-day thinking. It then goes into more complex decisions people make. For example, Ellenberg explains many misconceptions about lotteries and whether or not they can be mathematically beaten.

Ellenberg uses mathematics to examine real-world issues ranging from the loving of straight lines in the reporting of obesity to the game theory of missing flights, from the relevance to digestion of regression to the mean to the counter-intuitive Berkson's paradox.

International Collegiate Programming Contest (ICPC)

https://icpc.global

The International Collegiate Programming Contest is an algorithmic programming contest for college students. **Teams of three**, representing their university, work to solve the most real-world problems, fostering collaboration, creativity, innovation, and the ability to perform under pressure. Through training and competition, teams challenge each other to raise the bar on the possible. Quite simply, it is the oldest, largest, and most prestigious programming contest in the world.

If any group of three people is interested in participating, ***next year,***
read through the details and if you are still interested,
contact me about it.

Read and Watch

Topics 1: Binary Numbers, Floating Point Numbers, Random Numbers

1. https://docs.oracle.com/cd/E19957-01/806-3568/ncg_goldberg.html
2. Hougardy, Vygen: Chapter 2
3. https://www.youtube.com/watch?v=_tN2ev3hO14
4. https://www.youtube.com/watch?v=OxGsU8oIWjY
   (unless you (hopefully) know this already, still fun to watch)

Related

- https://en.wikipedia.org/wiki/Floating-point_arithmetic
- https://en.wikipedia.org/wiki/IEEE_754
- https://www.volkerschatz.com/science/float.html
- https://floating-point-gui.de
- https://en.wikipedia.org/wiki/Pseudorandom_number_generator

Topics 2:

- Read TRPL Chap 2 and TRPL Chap 3
- Hougardy, Vygen: Chapter 1 might be helpful.
- https://webhome.phy.duke.edu/~rgb/General/tao/tao.html

Student presentation

Definition:

Let $b \in \mathbb{N}, b \geq 2$, and $n \in \mathbb{N}$. Then there exist uniquely defined numbers $l \in \mathbb{N}$ and $z_i \in \{0, \dots, b-1\}$ for $i = 0, \dots, l-1$ with $z_{l-1} \neq 0$, such that

$$n = \sum_{i=0}^{l-1} z_i b^i .$$

The word $z_i \dots z_0$ is called the b-adic representation of n .

Example:

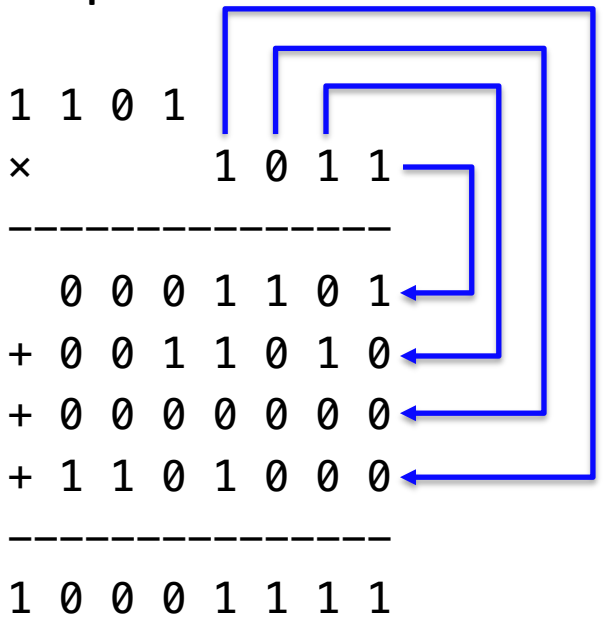
$$\begin{aligned} & 106 \\ & = 1 \cdot 10^2 + 0 \cdot 10^1 + 6 \cdot 10^0 \\ & = 1 \cdot 2^6 + 1 \cdot 2^5 + 0 \cdot 2^4 + 1 \cdot 2^3 + 0 \cdot 2^2 + 1 \cdot 2^1 + 0 \cdot 2^0 = 1101010_2 \\ & = 6 \cdot 16^1 + 10 \cdot 16^0 = 6\mathrm{A}_{16} \end{aligned}$$

Addition

```
Carry:  1 1 1
          ↓ ↓ ↓
        1 0 1 1
      + 1 1 0 1
      ---------
      1 1 0 0 0
```

Binary	Decimal
1011_2	11_{10}
1101_2	13_{10}
11000_2	24_{10}
10001111_2	143_{10}

Multiplication



2-Complement or how to represent negative numbers conveniently

Extend the size of the number by 1 digit and set it to 0, subtract 1 and flip all the bits.

Example:

$$6 = 110_2 \Rightarrow 1010_2 = -6$$
$$110 \Rightarrow 0110 - 0001 = 0101 \Rightarrow 1010$$

What is it good for?

Let us add 4 and -6 in binary:

$$0100 + 1010 = 1110$$

we flip

$$1110 \Rightarrow 0001$$

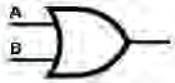
and add 1

$$0001 + 1 \Rightarrow 0010 = 2_{10}$$

Logic Gates

1.1. OR gate

The OR gate is one of the simplest gates. The symbol and truth table for the OR gate are shown in the illustration below. The output is 1 when *either* input A or B are 1, otherwise it is 0. The OR gate is represented by Boolean algebra operator $+$. Note that in Boolean algebra $1 + 1 = 1$.



OR gate

A	B	$A + B$
0	0	0
0	1	1
1	0	1
1	1	1

1.2. AND gate

The AND gate, is represented by the Boolean algebra operator '$\cdot$'. Its output is 1 when both inputs are 1, otherwise it is 0.

AND gate

A	B	$A \cdot B$
0	0	0
0	1	0
1	0	0
1	1	1

1.3. XOR gate

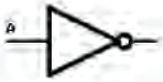
The eXclusive OR gate (XOR) is represented by the symbol $\oplus$.

XOR gate

A	B	$A \oplus B$
0	0	0
0	1	1
1	0	1
1	1	0

1.4. NOT gate

The inverter gate (NOT) represented by a bar above as in $\bar{A}$.

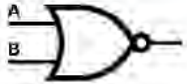


NOT gate

A	$\bar{A}$
0	1
1	0

1.5. NOR and NAND gate

At times, an OR gate is combined with an inverter to form Not-OR, or NOR for short. The table below shows this gate and its truth tables.



NOR gate

A	B	$\overline{A + B}$
0	0	1
0	1	0
1	0	0
1	1	0

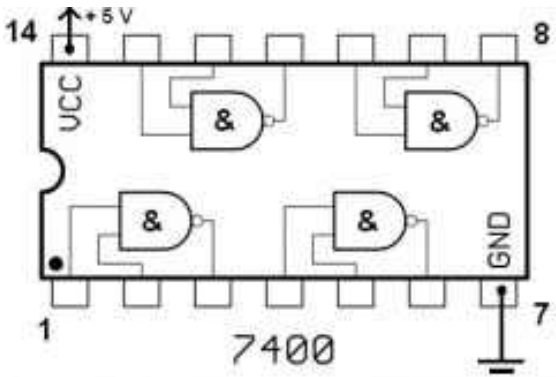
Other times, an AND gate is combined with an inverter to form to form Not-AND, or NAND for short.

NAND gate

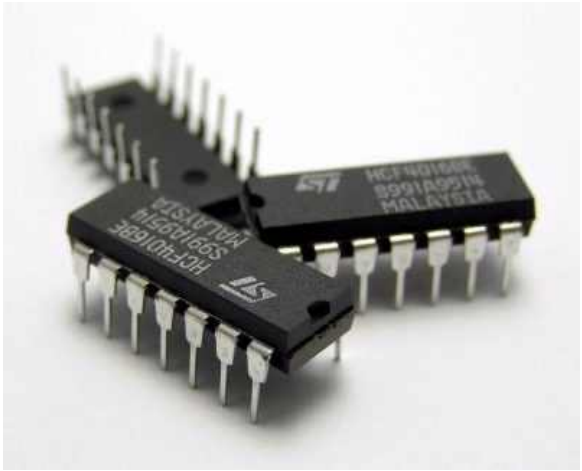
A	B	$\overline{A \cdot B}$
0	0	1
0	1	1
1	0	1
1	1	0

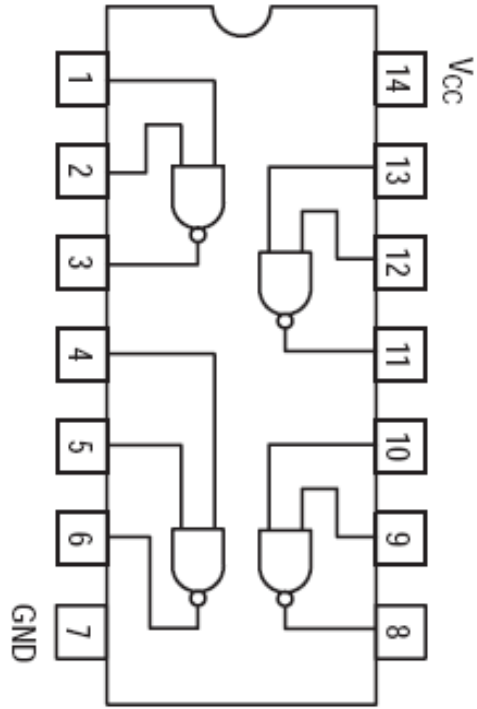
https://coertvonk.com/inquiries/computer-math/diode-resistor-logic-30701

https://en.wikipedia.org/wiki/List_of_7400-series_integrated_circuits



7400 Dual In-Line Package

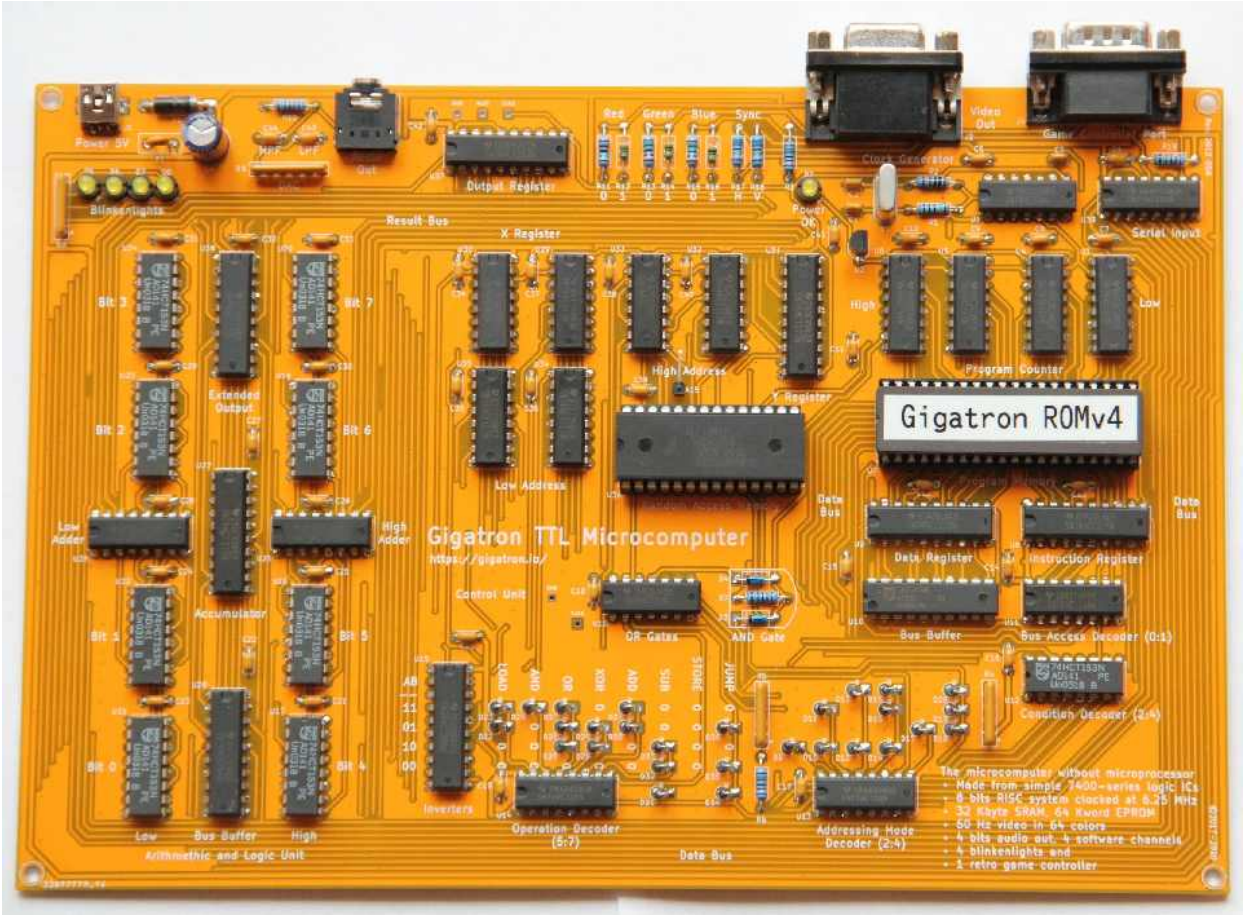




LS7400 Quad Dual Input NAND

Gigatron TTL Computer

https://www.jacob-pirna.de/category/ttl/gigatron



https://gigatron.io/?page_id=160

What is TTL?

Transistor-transistor logic (TTL) is a way of building logic gates on ICs, originally by using bipolar junction transistors for both switching (the first “T”) and for output buffering (the second “T”). This method was popularised by the Texas Instruments 7400-series that were introduced in the 1960s. These types of chips are among the first generations of ICs, before the invention of the microprocessor.

The 7400-family has all kinds of chips with individual NAND/NOR/AND/OR/XOR gates, inverters, buffers and chips with common blocks such as adders, shifters, multiplexers, decoders and flip-flops. They could be combined to build more complex systems, much like LEGO bricks.

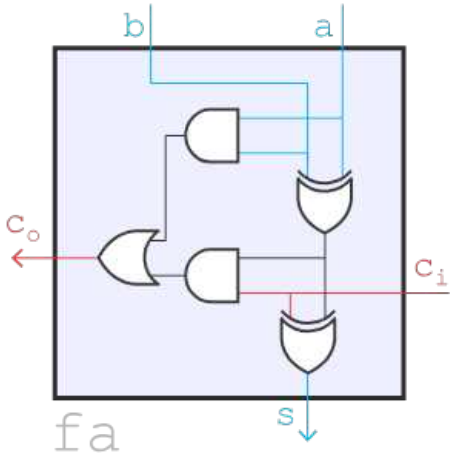
https://coertvonk.com/inquiries/computer-math/diode-resistor-logic-30701

The *full adder* forms the building block for all n-bit adders. This full adder adds the 1-bit values *a* and *b* to the incoming carry (c_i), and outputs a 1-bit sum (s) and a 1-bit outgoing carry (c_o). The carry is similar as when adding decimal numbers — if you have a carry from one column to the next, then that next column has to include that carry.

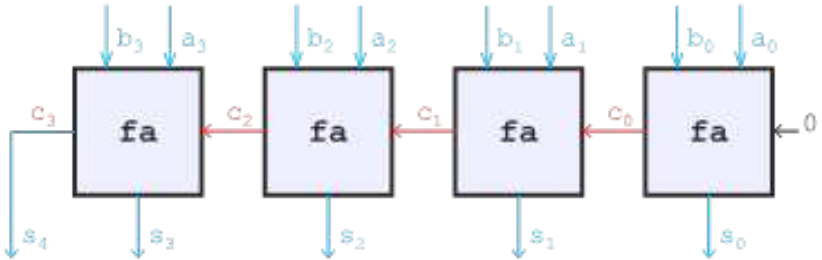
The truth table, shown below, gives the relation between these inputs and outputs. The Boolean equations and circuit are derived from this table.

Now that we can add 1-bit values, we can move on to n-bit values. The basic *carry-propagate adder* starts by adding the least significant bit and passing the carry on to each following bit. The *s* outputs are combined to form the sum *S*.

a	*b*	c_i	c_o	*s*
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1



The circuit shown below gives an example of a 4-bit carry-propagate adder. The carry has to propagate from the lowest to the highest bit position. This so-called “ripple carry” limits the speed of the circuit.



The Multiplier

https://coertvonk.com/inquiries/computer-math/diode-resistor-logic-30701

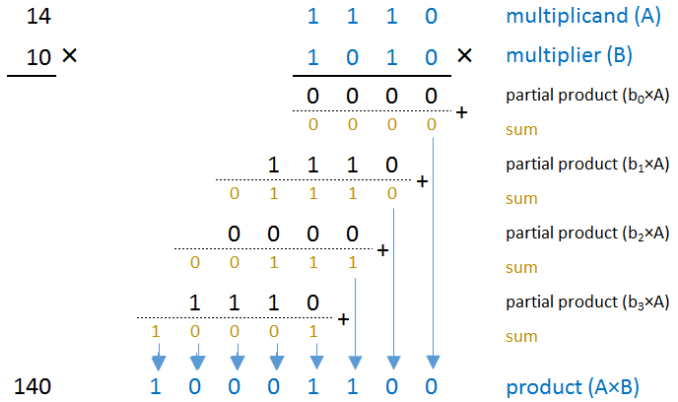
Multiplication is another important operation in digital signal processing (e.g. fast Fourier Transfers). It is an excellent example of combining simple logic functions to make a much more complex function. The two methods that allow us to do this called “array multipliers”.

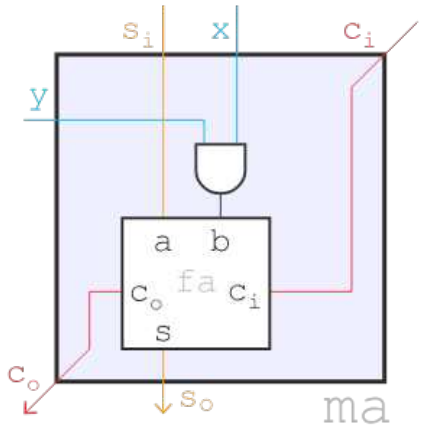
The array multiplier is a simple technique for implementing multiplication. It resembles the process how you would probably perform a multiplication yourself, aside from the fact that it sums up the partial products as it goes.

Each partial product bit position can be generated by a simple AND gate between corresponding positions of the multiplicand *A* and multiplier *B* bits. Adding the partial products forms the product.

The *multiplier adder* (ma) shown below, is the basic building block for the multiplier. It uses an AND gate to form the partial product of two binary digits. To calculate the partial sum and carry out signals, it reuses the 1-bit full adders described earlier. The Boolean equations describe the multiplier adder (ma) as a function of its ports:

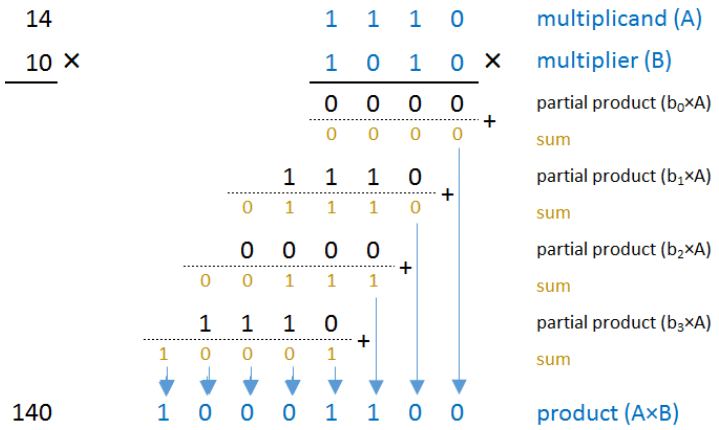
x	y	s_i	c_i	b	c_o	s_o
0	0	0	0	0	0	0
0	0	0	1	0	0	1
0	0	1	0	0	0	1
0	0	1	1	0	1	0
0	1	0	0	0	0	0
0	1	0	1	0	0	1
0	1	1	0	0	0	1
0	1	1	1	0	1	0
1	0	0	0	0	0	0
1	0	0	1	0	0	1
1	0	1	0	0	0	1
1	0	1	1	0	1	0
1	1	0	0	1	0	1
1	1	0	1	1	1	1
1	1	1	0	1	1	0
1	1	1	1	1	1	0

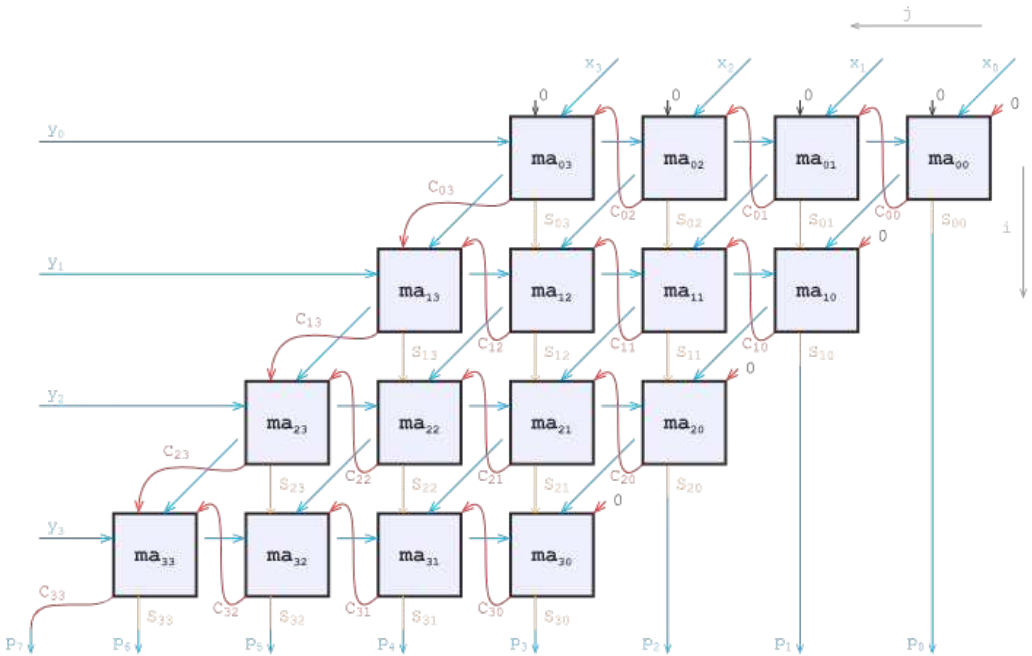




Carry Propagate Array Multiplier

The *carry propagation array multiplier* works similarly to the way humans multiply. The ma_{ij} building blocks connect in a grid, where ij identifies the row and column. The figure below gives an example of a 4-bit multiplier.





https://coertvonk.com/inquiries/computer-math/diode-resistor-logic-30701

```
1  let mut n: usize = 0;
2  let mut e: f64   = 1.0;
3  let mut s        = [0.0; 2048]; s[0] = 1.0 + e;
4
5  while e       > 0.0   // Alternative 1
6  while e + 1.0 > 1.0   // Alternative 2
7  while s[n]    > 1.0   // Alternative 3
8     { n = n + 1; e = e / 2.0; s[n] = 1.0 + e; }
```

Does the loop terminate?

Will the program crash?

In case it terminates will *n* have the same value in all alternatives? (Lines 5,6,7)

```
1  let mut n: usize = 0;
2  let mut e: f64   = 1.0;
3  let mut s        = [0.0; 2048]; s[0] = 1.0 + e;
4
5  while e        > 0.0   // n = 1075
6  while e + 1.0 > 1.0   // n = 53
7  while s[n]     > 1.0   // n = 53
8     { n = n + 1; e = e / 2.0; s[n] = 1.0 + e; }
```

The loop will terminate, the program will not crash,
and *n* is different in most cases, depending on
the architecture, the compiler, and the compiler options.

“God made the integers, all else is the work of man.”

—*Leopold Kronecker* (1886)

„Die ganzen Zahlen hat der liebe Gott gemacht, alles andere ist Menschenwerk.“

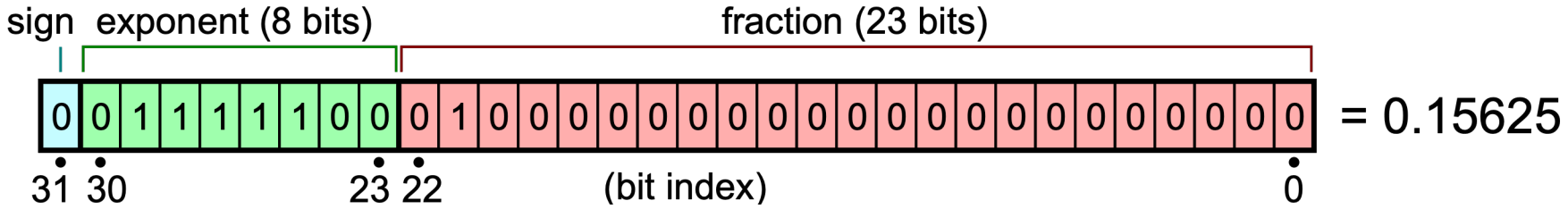
Hilberts Hotel: https://www.youtube.com/watch?v=OxGsU8oIWjY

Question: Given 1 GB memory, what is the largest integer number we can store?

$$1\ \text{GB} \cong 1024 \cdot 1024 \cdot 1024 \cdot 8 = 2^{33} \implies 2^{2^{33}} = 2^{8589934592} = 10^{2585827972}$$

However, not all numbers can be represented well numerically in a computer.

Symb	Name	Computer
$\mathbb{N}$	natural numbers, e.g., 1, 2, 666	✓
$\mathbb{Z}$	integer numbers, e.g., 1, 2, -666	✓
$\mathbb{Q}$	rational numbers, e.g., ½ , -¼, 2, -111	✓
$\mathbb{R}$	real numbers, e.g., $\sqrt{2}, \pi, {}^{1}/_{3}, 7, -1024$	—
$\mathbb{C}$	complex numbers, e.g., $-1 + 3i$	—
	$\mathbb{N} \subset \mathbb{Z} \subset \mathbb{Q} \subset \mathbb{R} \subset \mathbb{C}$	



$$= (1-)^{b_{31}} \times 2^{(E-127)} \times \left(1 + \sum_{i=1}^{23} b_{23-i} 2^{-i}\right) = (+1) \times 2^{-3} \times 1.25 = +0.15625$$

In this example:

- $\text{sign} = b_{31} = 0$,
- $(-1)^{\text{sign}} = (-1)^0 = +1 \in \{-1, +1\}$,
- $E = (b_{30} b_{29} \dots b_{23})_2 = \sum_{i=0}^{7} b_{23+i} + 2^{+i} = 124 \in \{1, \dots, (2^8 - 1) - 1\} = \{1, \dots, 254\}$,
- $2^{(E-127)} = 2^{124-127} = 2^{-3} \in \{2^{-126}, \dots, 2^{127}\}$,
- $1.\, b_{22} b_{21} \dots b_0 = 1 + \sum_{i=1}^{23} b_{23-i} 2^{-i} = 1 + 1 \cdot 2^{-2} = 1.25 \in \{1{,}1 + 2^{-23}, \dots, 2 - 2^{-23}\}$

Note the implicit 1 !

https://en.wikipedia.org/wiki/Single-precision_floating-point_format

Decoding the Exponent

Exponent	fraction = 0	fraction ≠ 0	Equation
$00_H = 00000000_2$	±zero	subnormal number	$(-1)^{\text{sign}} \times 2^{-126} \times 0.\text{fraction}$
01_H, ..., $FE_H = 00000001_2$, ..., 11111110_2	normal value		$(-1)^{\text{sign}} \times 2^{\text{exponent}-127} \times 1.\text{fraction}$
$FF_H = 11111111_2$	±infinity	NaN (quiet, signaling)	

Type	Bits				Exponent bias	Bits precision	Number of decimal digits
	Sign	Exponent	Significand	Total			
Half (IEEE 754-2008)	1	5	10	16	15	11	~3.3
Single	1	8	23	32	127	24	~7.2
Double	1	11	52	64	1023	53	~15.9
x86 extended precision	1	15	64	80	16383	64	~19.2
Quad	1	15	112	128	16383	113	~34.0

Associative and distributive law do not hold for floating point numbers

```rust
let a = 0.1;
let b = 0.2;
let c = 0.3;

let add_lhs = (a + b) + c;
let add_rhs = a + (b + c);
println!("(a + b) + c = {add_lhs}"); // 0.6000000000000001
println!("a + (b + c) = {add_rhs}"); // 0.6

let mul_lhs = (a * b) * c;
let mul_rhs = a * (b * c);
println!("(a * b) * c = {mul_lhs}"); // 0.006000000000000001
println!("a * (b * c) = {mul_rhs}"); // 0.006
```

Associative and distributive law do not hold for floating point numbers

```
let a : f32 = 1e38;
let b : f32 = -1e38;
let c : f32 = 1.0;

let add_lhs = (a + b) + c;
let add_rhs = a + (b + c);
println!("(a + b) + c = {add_lhs}"); // 1
println!("a + (b + c) = {add_rhs}"); // 0
```

What is the most accurate way to sum the list of signed numbers (1e-10, 1e10, -1e10) when we have only 8 significant digits ?

(1) Just add the numbers in the given order

(2) Sort the number numerically decreasing, then add in sorted order

(3) Sort the numbers numerically increasing, then add in sorted order

(4) Sort the numbers numerically, then pick off the largest of values coming from either end, repeating the process until done

What is the most accurate way to sum the list of signed numbers (1e-10, 1e10, -1e10) when we have only 8 significant digits ?

(1) Just add the numbers in the given order

(2) Sort the number numerically decreasing, then add in sorted order

(3) Sort the numbers numerically increasing, then add in sorted order

(4) Sort the numbers numerically, then pick off the largest of values coming from either end, repeating the process until done (or equivalently sort by absolute value.)

Quite a lot of people, including scientists, use floating-point numbers assuming they are a faithful representation of mathematical real numbers. This assumption lives in the dangerous grey zone between outrageously false and actually true — it is true frequently enough that those times when it is not come as nasty surprises. This page intends to spread some knowledge about floating-point arithmetic that, while not very deep, is less widespread than it ought to be.

Representation

The following trivia about floating-point representation will hopefully keep you from ever seeing them as faithful to reals again:

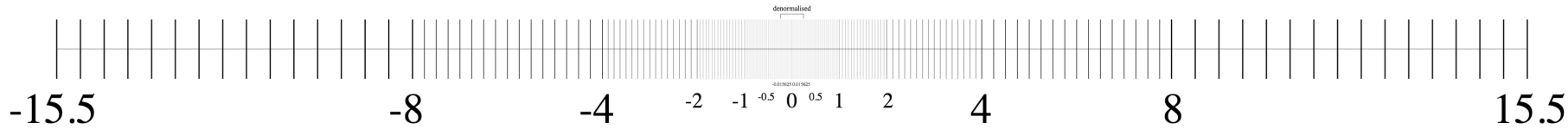
There are only a finite number of different floats

While this is obvious, it puts paid to the idea of representing reals in general. Since the representation basically consists of storing integer or fractional digits in a given base, in fact all representable numbers are rationals.

Floats are discrete but not equidistant

Also a bit of a triviality. Any finite subset of the reals is discrete. Unlike integers, floats are dense close to zero, and rather thin on the ground at large moduli.

The diagram below shows the location of the finite numbers representable by a hypothetical 8-bit floating-point format that has a 4-bit mantissa (with an implied fifth leading 1 bit) and a 3-bit exponent but otherwise follows IEEE conventions.



You can see that the spacing between adjacent numbers jumps by a factor of two at every power of two larger than 0.25. 0.25 is the smallest normalised number in this representation (by modulus), and numbers between -0.25 and 0.25 are denormalised, indicated by the square bracket above the axis. The ±0.015625 printed above the zero are the smallest non-zero representable numbers. The zero exponent of subnormal (denormalised) numbers implies the same power-of-two multiplier as an exponent of one, or there would be a gap because subnormals lack the implied leading one bit.

Only (reduced) fractions with power-of-two denominators are representable accurately

Yes, really! This is because any other denominator gives rise to an infinite number of fractional digits in a binary representation, which of course cannot all be stored. Do you feel your expectations of floats coming down to meet reality?

Non-negative floats sort like integers

This is less well known. To make it clearer: If you interpret two non-negative floating-point numbers as integers with the same size and endianness, the result of a comparison between those integers will be the same as that of the floating-point comparison. This is a consequence of the fact that the exponent is located in higher-order bits than the significand, and that the implied leading one bit of the significand does not affect comparisons. This does not account for NaNs (though it holds for positive infinity).

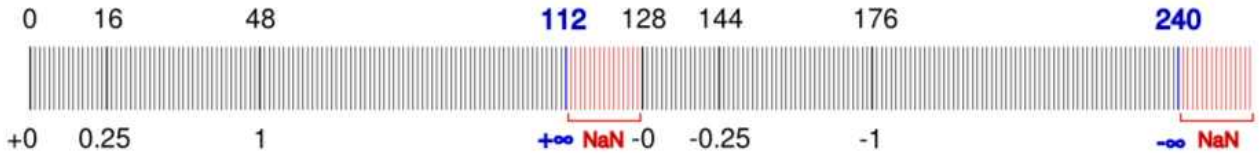
Negative floats have the inverse sorting order of the corresponding negative integers.

This has the interesting implication that under the restrictions above, it is possible to find the upper or lower neighbour of a floating-point variable by an integer increment or decrement:

```
++ *(int64_t*) &double_variable;    /* successor */
-- *(int32_t*) &float_variable;     /* predecessor */
```

By handling negative numbers properly, one could write functions that return the successor or predecessor of any floating-point value. Indeed, the D programming language provides such methods♂ for floating-point types.

The figure below shows the correspondence between our hypothetical 8-bit floating-point type and unsigned integers.



As you can see, the positive and negative float values are monotonically increasing with their integer counterparts. ±0.25 is the smallest normalised number by modulus, as noted above. There are more numbers above 1 (modulus) than below, because 1 is the smallest number with the middle-of-the-range exponent. The largest representable exponent indicates either infinity (for zero mantissa) or NaN, so the chance of hitting a NaN when generating data at random is rather higher than one would expect.

Take care converting large integers

64-bit integers may not fit into a `double`. Unless you know only 53 bits are used, the only suitable floating-point type is the 80-bit extended precision format[W] (long double) on x86 processors.

Careful with iterations

In many if not most cases numerical computations involve iteration rather than just the evaluation of a single formula. This can cause errors to accumulate, as the chance of an intermediate result not being exactly representable rises. Even if the target solution is an attractive fixed point of your iteration formula, numerical errors may catapult you out of its domain of attraction.

Rounding

In addition to the rounding errors introduced at every step in a computation, the values your processor computes for transcendental functions may be off by more than basic arithmetic. This is because optimal rounding to the nearest representable number requires distinguishing on which side of the middle between them (an infinitely narrow line) the true result lies. Transcendental functions are power series with infinitely many non-zero coefficients, so one cannot really know that for a given argument without computing an infinity of terms of the series.

Non-power-of-two integer factors between fractional numbers are inexact

In the previous section, we learnt that most fractional numbers cannot be represented exactly. In addition, fractional numbers differing by an integral factor that is not a power of two will have a different relative error (as at least one of them will not be exactly representable). This means that choosing different units for the input to your calculation, such as metres versus millimetres, may get you a different result. Equality comparisons between quantities computed from input differing in that way may fail.

Additions are more problematic than multiplications

Floating-point multiplications are pretty much optimal, with the significands being multiplied and the exponents added, and the surplus least significant bits being discarded. An addition, by contrast, may have no effect at all if the numbers are too different. The reason for this lies in the fact that the distribution of representable floating-point values resembles a logarithmic scale more than a linear one (see the first figure above).

As a consequence, the simplest problem to provide unexpected results is adding up a series of numbers by the naive algorithm of successively adding them to a running sum variable. As the sum becomes larger, additions become increasingly inaccurate. The result can therefore depend on the order of the numbers. For addends of similar size, creating an adder tree (adding every two values, then recursively every two sums) is a possible fix, but rather complex to program. The following section will demonstrate an easier and more general solution.

Even more than simple addition, the computation of the mean and standard deviation shows the incompatibility of naive algorithms with floating-point numbers. For genuine real numbers the following formulas yield the correct result:

$$\begin{aligned} \bar{x} &= \frac{1}{N}\sum x_n \\ \sqrt{\overline{(x-\bar{x})^2}} &= \frac{1}{N}\sqrt{N\sum x_n^2 - \left(\sum x_n\right)^2} \end{aligned}$$

Computing the standard deviation from the sum and the sum of squares with floating-point numbers will however frequently get you a NaN, arising from a negative term in the square root, which in turn is caused by inaccuracies in the summation. Again, see the following section for a solution.

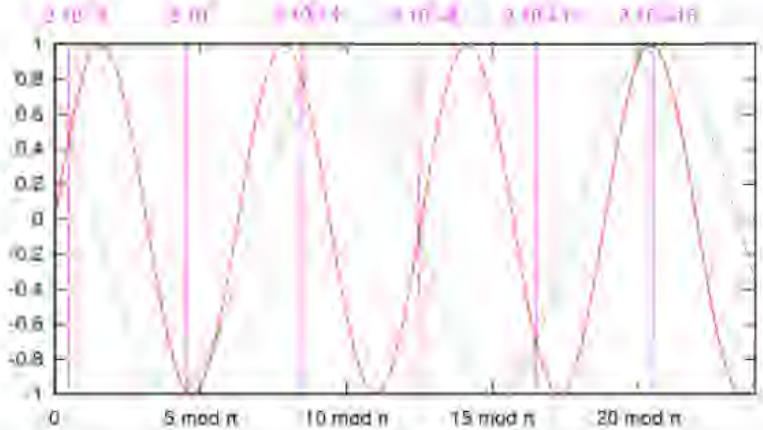
Sampling effects for trigonometric functions

https://www.volkerschatz.com/science/float.html

The value of the sine function at 0 is 0, even numerically. But what is the next largest argument for which the numerical `sin()` returns zero? Actually I do not know either, but the number is certainly huge compared to 1. For double-precision floats finding this number would probably require years of computation time; for single precision there is no argument yielding 0 below the point where numerical trigonometric functions become meaningless. Run this program yourself to check.

The most readily noticed implication of this is that rotation matrices that involve right angles have small non-zero entries where there should be zeros. This is due to the numerical $\cos \pi/2$ differing slightly from zero. (For the 1 entries, the corresponding discrepancy does not show up because of rounding.) Generally, the periodicity of trigonometric functions is hidden by the fact that differences between floating-point numbers are always an irrational factor away from their period, 2π.

What was this remark in the first paragraph about trigonometric functions becoming meaningless? When sampling functions uniformly, the highest frequency has to be below the Nyquist limit (half the sampling rate) for the function to be representable adequately. Floating-point numbers sample the real axis in a far from uniform manner (see the first figure again!). At some point the difference between adjacent floating-point numbers becomes larger than π — the equivalent of exceeding the Nyquist limit in uniform sampling.



This figure shows the sine in red at the scale of large floating-point numbers around $2 \cdot 10^{16}$. The pink lines are successive numbers exactly representable in double precision. The dotted green curve shows a sine with a period of $2\pi / |\pi/2 - 1| \approx 11$ that has the same value as sin(x) at all double-precision floating point numbers of this size or larger. The figure was non-trivial to create. I used this program to find the successor of a floating-point number and this Perl script to compute the remainder mod 2π of any number exactly, to find out the position of representable numbers relative to the sine.

Let's start with the simplest example above, summation of a number series. A more accurate algorithm is the following (Kahan summation):

```
for all elements x in the number series
    old_sum := sum
    x := x - delta
    sum := sum + x
    delta := (sum - old_sum) - x
```

The method works as follows: The amount lost when updating the sum variable is computed by an expression undoing the summation, which would always result in 0 mathematically but does not because of numerical errors. Note the parentheses. This discrepancy is subtracted in the next run of the loop from the next element to be added, which unlike the sum is small enough for the subtraction to have an effect.

The important thing to take away from this is that numerical errors are not noise introduced at random in a computation, but can be quantified and possibly compensated.

[Aside: The Kahan summation algorithm can still fail when there are cancellations much larger than the correct end result. See here for even more robust algorithms, a test case and link to a paper.]

A similar algorithm exists for computing the mean and standard deviation. This time `delta` is local to the loop body, but the other variables have to be initialised to 0 as above:

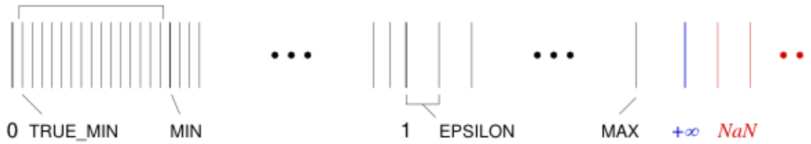
```
for all elements x in the number series
    count := count + 1
    delta := x - mean
    mean := mean + delta / count
    aux := aux + delta * (x - mean)
stddev := sqrt(aux / count)
```

http://webmail.cs.yale.edu/publications/techreports/tr222.pdf

This method is attributed to Welford and others. This paper contains a comparison of algorithms for computing the variance from a numeric point of view, among others the one that yields the above for the standard deviation (equation 1.3).

Floating-point type property constants

The standard C header file `float.h` contains preprocessor constants describing the details of your floating-point implementation, such as the size of the mantissa, the range of exponents, the maximum and minimum representable numbers, and others. Some of the constants are special representable numbers and can be marked on a simplified version of the scale above:



Normally the constants are prefixed with one of `FLT_`, `DBL_` or `LDBL_` for `float` (single precision), `double` or `long double`. I have left the prefix out for the purpose of presentation. The values of the most important constants for our 8-bit floating-point type are:

Constant	Value	Explanation
EPSILON	0.0625	Difference between 1 and next larger representable value
MIN	0.25	Smallest normalised representable value
MAX	15.5	Largest positive representable value
MANT_DIG	5	Mantissa bit size **including implied leading 1**
MIN_EXP	-2	Minimum exponent for normalised number; also implied for subnormals
MAX_EXP	3	Maximum exponent
TRUE_MIN	0.015625	Smallest positive representable value

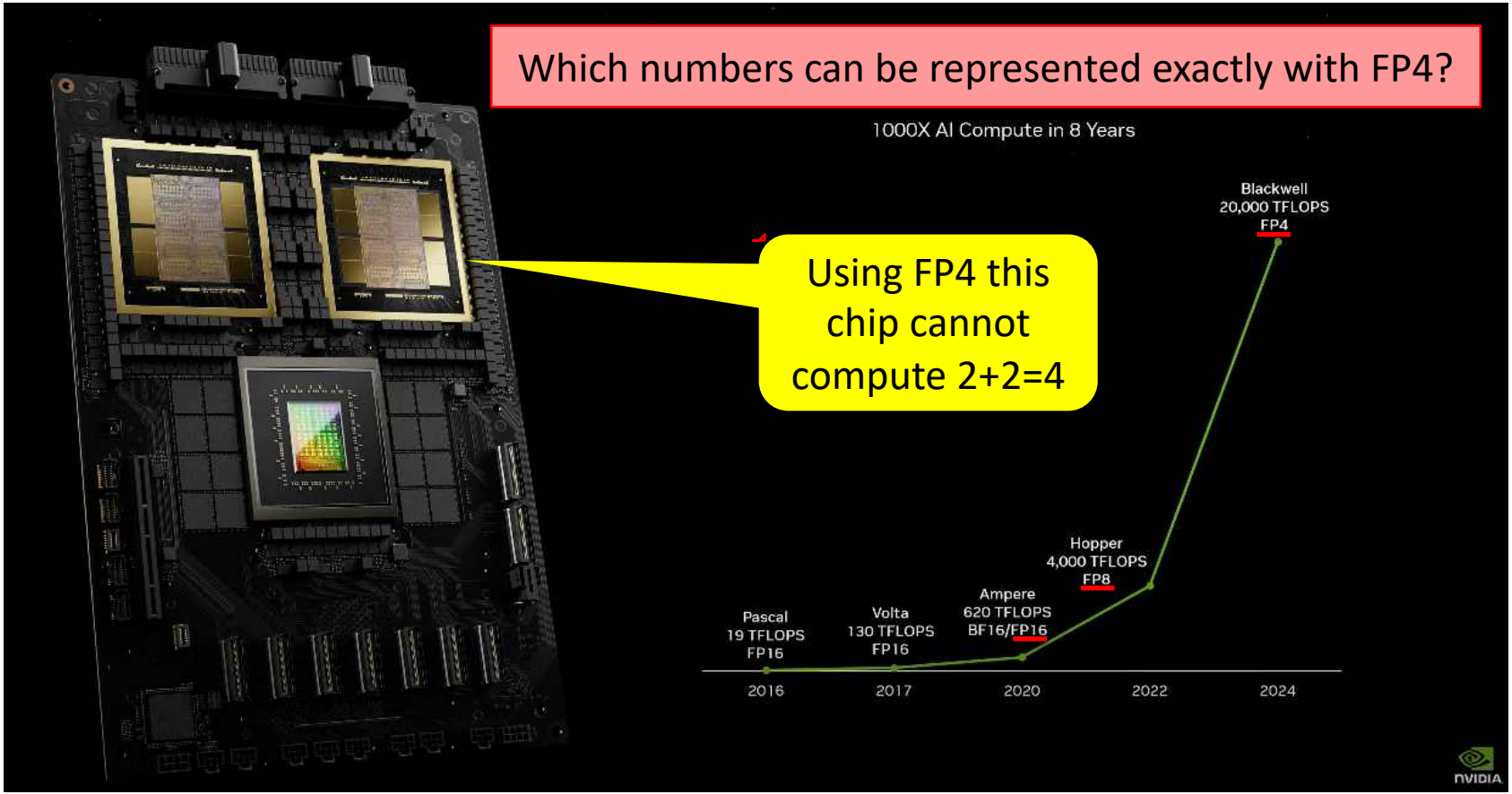
Note that the number of mantissa bits `MANT_DIG` includes the leading one bit, regardless of whether it is explicitly stored or not. Similarly, `TRUE_MIN` is the smallest representable value which is usually a subnormal, but not necessarily. The constant `HAS_SUBNORM` indicates if subnormals are supported at all. If this is false (zero), `TRUE_MIN` will presumably be equal to `MIN`.

The IEEE 754 standard defines four rounding modes:

- **Round to nearest (RN):** The result of the arithmetic operation is rounded to the closest representable value. If the result is exactly halfway between two representable values, the one whose least significant bit is even is chosen.
- **Round towards ∞ (RU):** The result of an arithmetic operation is rounded upward, i.e., to the closest representable value in the direction of ∞.
- **Round towards −∞ (RD):** The result of an arithmetic operation is rounded down- ward, i.e., to the closest representable value in the direction of −∞.
- **Round towards zero (RZ):** The result of an arithmetic operation is rounded toward zero.

See also https://en.wikipedia.org/wiki/Rounding

Trading generality for performance in Hardware: NVIDIA Broadwell



FP4

```
-Inf
-3.0
-2.0
-1.5
-1.0
-0.5
 0.0
 0.5
 1.0
 1.5
 2.0
 3.0
 Inf
 NaN

That's
 all
folks!
```

$$r = 1, A = \pi r^2 = \pi, \alpha_n = \frac{2\pi}{n}$$

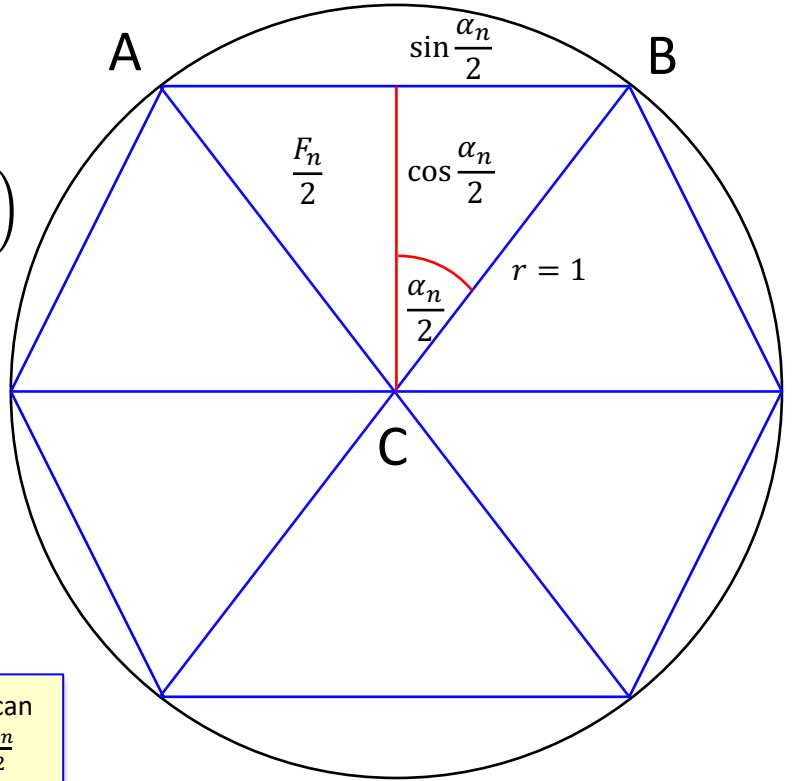
$$F_n = \cos\frac{\alpha_n}{2}\sin\frac{\alpha_n}{2}$$

$$A_n = nF_n = \frac{n}{2}\left(2\cos\frac{\alpha_n}{2}\sin\frac{\alpha_n}{2}\right) = \frac{n}{2}\sin\alpha_n = \frac{n}{2}\sin\left(\frac{2\pi}{n}\right)$$

$$\sin\frac{\alpha_n}{2} = \sqrt{\frac{1-\cos\alpha_n}{2}} = \sqrt{\frac{1-\sqrt{1-\sin^2\alpha_n}}{2}}$$

$$\alpha_6 = 60°, \sin\alpha_6 = \frac{\sqrt{3}}{2}, F_6 = \frac{\sqrt{3}}{4}, A_6 = 3\frac{\sqrt{3}}{2}$$

if we have $\sin\alpha$ we can now compute $\sin\frac{\alpha_n}{2}$



First try

```rust
use std::f64::consts::PI;

fn print(n: u32, area: f64, sine: f64) {
    println!("{n:14} {area:22.15} {:22.15} {sine:22.15}", area - PI);
}

fn main() {
    let mut sine = f64::sqrt(3.0) / 2.0; // = sin(60°) = sin(2pi/6)
    let mut area = 3.0 * sine;           // = 6/2 sin(2pi/6)
    let mut n    = 6;

    print(n, area, sine);

    while sine > 1e-10 {
        sine  = f64::sqrt((1.0 - f64::sqrt(1.0 - sine * sine)) / 2.0);
        area  = (n as f64) * sine; // n=2n => n/2 sin => n sin
        n    *= 2;
        print(n, area, sine);
    }
}
```

n	computed π	error	$\sin(360/n)$
6	2.598076211353316	-0.543516442236477	0.866025403784439
12	3.000000000000000	-0.141592653589794	0.500000000000000
24	3.105828541230250	-0.035764112359543	0.258819045102521
48	3.132628613281237	-0.008964040308556	0.130526192220052
96	3.139350203046872	-0.002242450542921	0.065403129230143
192	3.141031950890530	-0.000560702699263	0.032719082821776
384	3.141452472285344	-0.000140181304449	0.016361731626486
768	3.141557607911622	-0.000035045678171	0.008181139603937
1536	3.141583892148936	-0.000008761440857	0.004090604026236
3072	3.141590463236762	-0.000002190353031	0.002045306291170
6144	3.141592106043048	-0.000000547546745	0.001022653680353
12288	3.141592516588155	-0.000000137001638	0.000511326906997
24576	3.141592618640789	-0.000000034949004	0.000255663461803
49152	3.141592645321216	-0.000000008268577	0.000127831731987
98304	3.141592645321216	-0.000000008268577	0.000063915865994
196608	3.141592645321216	-0.000000008268577	0.000031957932997
393216	3.141592645321216	-0.000000008268577	0.000015978966498
786432	3.141593669849427	0.000001016259634	0.000007989485855
1572864	3.141592303811738	-0.000000349778055	0.000003994741190
3145728	3.141608696224804	0.000016042635011	0.000001997381017
6291456	3.141586839655041	-0.000005813934752	0.000000998683561
12582912	3.141674265021758	0.000081611431964	0.000000499355676
25165824	3.141674265021758	0.000081611431964	0.000000249677838
50331648	3.143072740170040	0.001480086580246	0.000000124894489
100663296	3.159806164941135	0.018213511351342	0.000000062779708
201326592	3.181980515339464	0.040387861749671	0.000000031610136
402653184	3.354101966249685	0.212509312659892	0.000000016660005
805306368	4.242640687119286	1.101048033529493	0.000000010536712
1610612736	6.000000000000000	2.858407346410207	0.000000007450581
3221225472	0.000000000000000	-3.141592653589793	0.000000000000000

First try: What went wrong?

```rust
use std::f64::consts::PI;

fn print(n: u32, area: f64, sine: f64) {
    println!("{n:14} {area:22.15} {:22.15} {sine:22.15}", area - PI);
}

fn main() {
    let mut sine = f64::sqrt(3.0) / 2.0; // = sin(60°) = sin(2pi/6)
    let mut area = 3.0 * sine;           // = 6/2 sin(2pi/6)
    let mut n    = 6;

    print(n, area, sine);

    while sine > 1e-10 {
        sine  = f64::sqrt((1.0 - f64::sqrt(1.0 - sine * sine)) / 2.0);
        area  = (n as f64) * sine; // n=2n => n/2 sin => n sin
        n    *= 2;
        print(n, area, sine);
    }
}
```

$$1 - \sqrt{1 - \varepsilon^2}$$

$$\varepsilon = \sin \alpha_n$$

$$\alpha_n \to 0: \quad \sin \alpha_n \to 0$$

$$r = 1, A = \pi r^2 = \pi$$

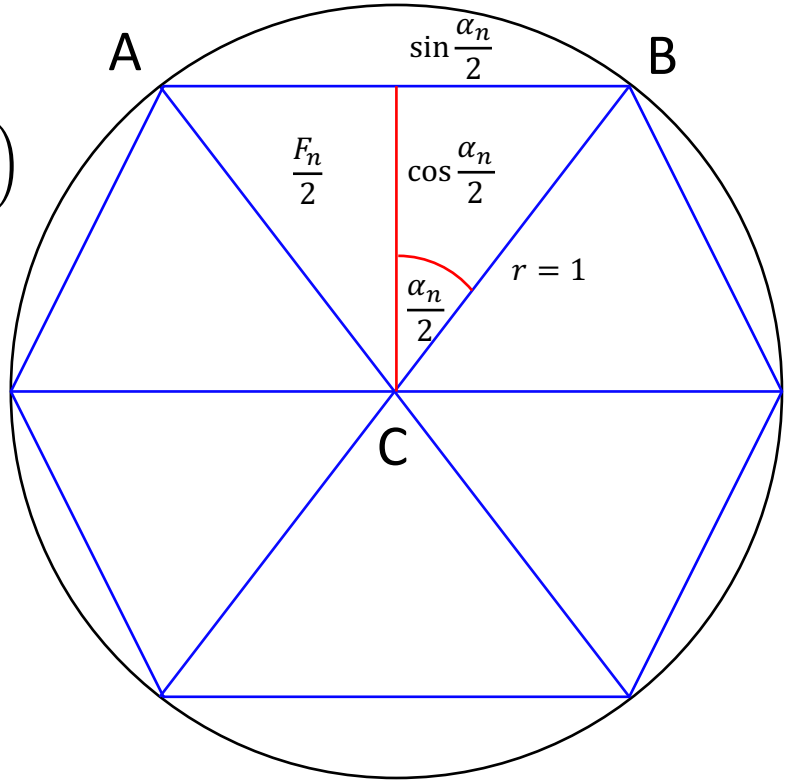
$$F_n = \cos\frac{\alpha_n}{2}\sin\frac{\alpha_n}{2}$$

$$A_n = nF_n = \frac{n}{2}\left(2\cos\frac{\alpha_n}{2}\sin\frac{\alpha_n}{2}\right) = \frac{n}{2}\sin\alpha_n = \frac{n}{2}\sin\left(\frac{2\pi}{n}\right)$$

$$\sin\frac{\alpha_n}{2} = \sqrt{\frac{1-\cos\alpha_n}{2}} = \sqrt{\frac{1-\sqrt{1-\sin^2\alpha_n}}{2}}$$

$$= \frac{\boldsymbol{\sin\alpha_n}}{\sqrt{\boldsymbol{2\left(1+\sqrt{1-\sin^2\alpha_n}\right)}}}$$

$$\alpha_6 = 60°, \sin\alpha_6 = \frac{\sqrt{3}}{2}, F_6 = \frac{\sqrt{3}}{4}, A_6 = 3\frac{\sqrt{3}}{2}$$



Try 2

```rust
use std::f64::consts::PI;

fn print(n: u32, area: f64, sine: f64) {
    println!("{n:14} {area:22.15} {:22.15} {sine:22.15}", area - PI);
}

fn main() {
    let mut old_area = 0.0;
    let mut sine     = f64::sqrt(3.0) / 2.0; // = sin(60°) = sin(2pi/6)
    let mut area     = 3.0 * sine;           // = 6/2 sin(2pi/6)
    let mut n        = 6;

    print(n, area, sine);

    while area > old_area {
        old_area = area;
        sine     = sine / f64::sqrt(2.0 * (1.0 + f64::sqrt((1.0 + sine) * (1.0 - sine))));
        area     = (n as f64) * sine; // n=2n => n/2 sin => n sin
        n       *= 2;
        print(n, area, sine);
    }
}
```

n	computed π	error	$\sin(360/n)$
6	2.598076211353316	-0.543516442236477	0.866025403784439
12	3.000000000000000	-0.141592653589793	0.500000000000000
24	3.105828541230249	-0.035764112359544	0.258819045102521
48	3.132628613281238	-0.008964040308555	0.130526192220052
96	3.139350203046867	-0.002242450542926	0.065403129230143
192	3.141031950890509	-0.000560702699284	0.032719082821776
384	3.141452472285462	-0.000140181304332	0.016361731626487
768	3.141557607911857	-0.000035045677936	0.008181139603937
1536	3.141583892148318	-0.000008761441475	0.004090604026235
3072	3.141590463228050	-0.000002190361744	0.002045306291164
6144	3.141592105999271	-0.000000547590522	0.001022653680338
12288	3.141592516692156	-0.000000136897637	0.000511326907014
24576	3.141592619365383	-0.000000034224410	0.000255663461862
49152	3.141592645033690	-0.000000008556103	0.000127831731976
98304	3.141592651450766	-0.000000002139027	0.000063915866118
196608	3.141592653055036	-0.000000000534757	0.000031957933076
393216	3.141592653456104	-0.000000000133690	0.000015978966540
786432	3.141592653556371	-0.000000000033422	0.000007989483270
1572864	3.141592653581438	-0.000000000008355	0.000003994741635
3145728	3.141592653587705	-0.000000000002089	0.000001997370818
6291456	3.141592653589271	-0.000000000000522	0.000000998685409
12582912	3.141592653589663	-0.000000000000130	0.000000499342704
25165824	3.141592653589761	-0.000000000000032	0.000000249671352
50331648	3.141592653589786	-0.000000000000008	0.000000124835676
100663296	3.141592653589791	-0.000000000000002	0.000000062417838
201326592	3.141592653589794	0.000000000000000	0.000000031208919
402653184	3.141592653589794	0.000000000000001	0.000000015604460
805306368	3.141592653589794	0.000000000000001	0.000000007802230

Random Numbers

Being Deterministic: https://en.wikipedia.org/wiki/Deterministic_algorithm

https://xkcd.com/221

```
int getRandomNumber()
{
    return 4;  // chosen by fair dice roll.
               // guaranteed to be random.
}
```

"Insanity Is Doing the Same Thing Over and Over Again and Expecting Different Results."

— unknown

An algorithm is deterministic, if **given a particular input, it will always produce the same output**, with the underlying machine always passing through the same sequence of states. https://en.wikipedia.org/wiki/Deterministic_algorithm

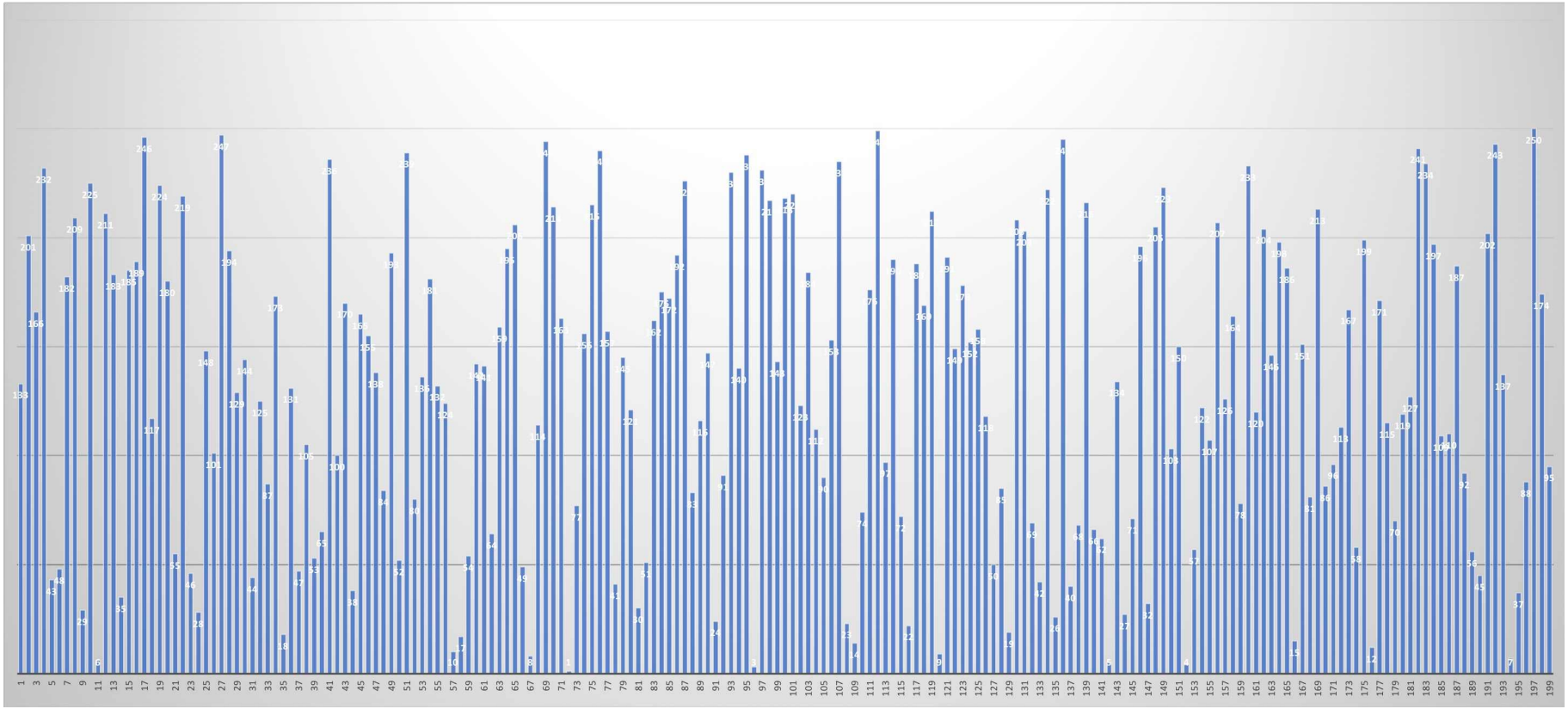
A deterministic algorithm computes a mathematical function; a function has a unique value for any input in its domain, and the algorithm is a process that produces this particular value as output.

Non-determinism can result, for example, from:

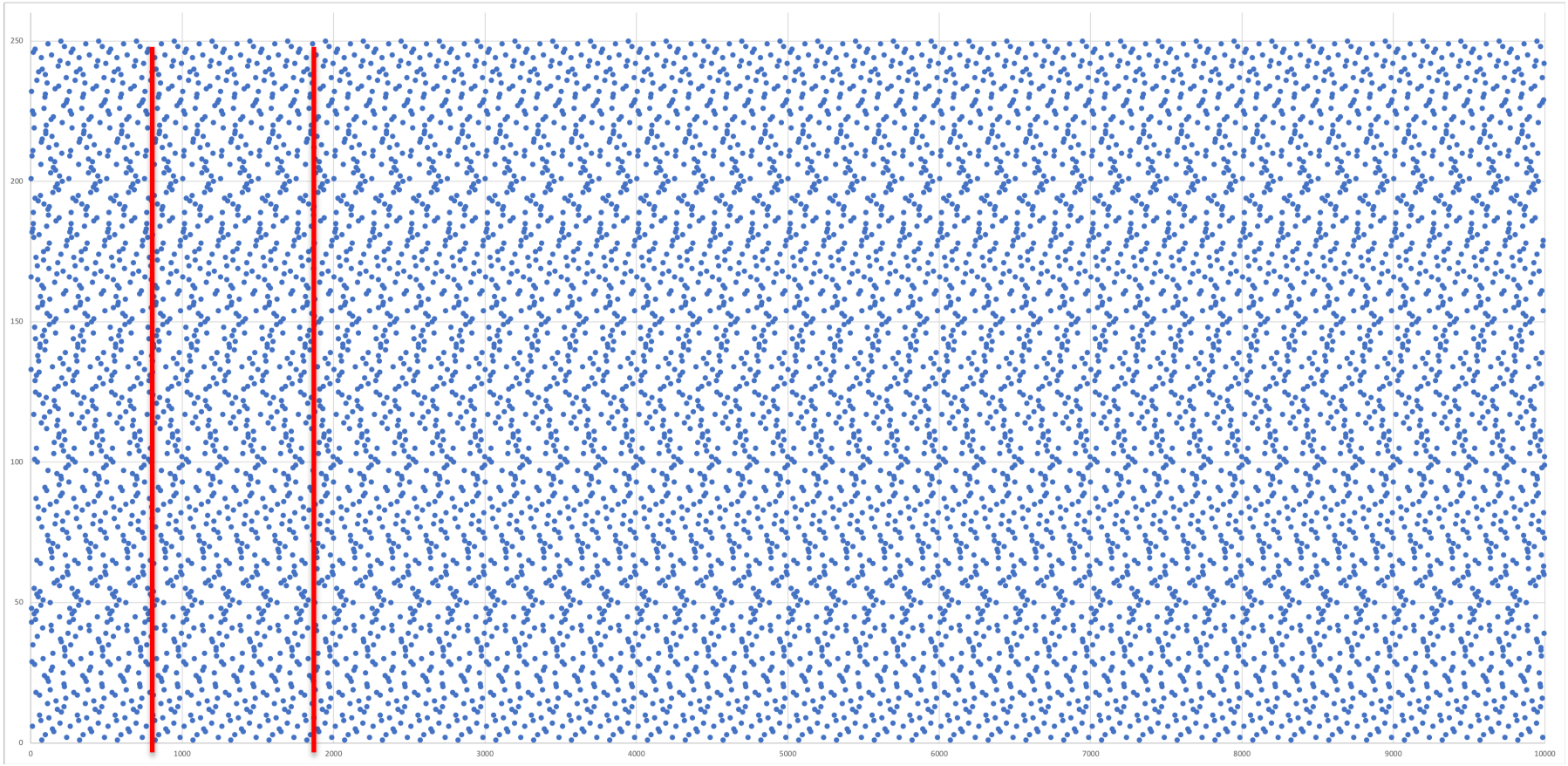
- use of an external state other than the input, such as a hardware timer value.
- if multiple processors writing to the same data at the same time. In this case, the precise order in which each processor writes its data will affect the result.

While digital computers are thought of being deterministic,
Quantum computers a probabilistic, i.e., non-deterministic by definition.

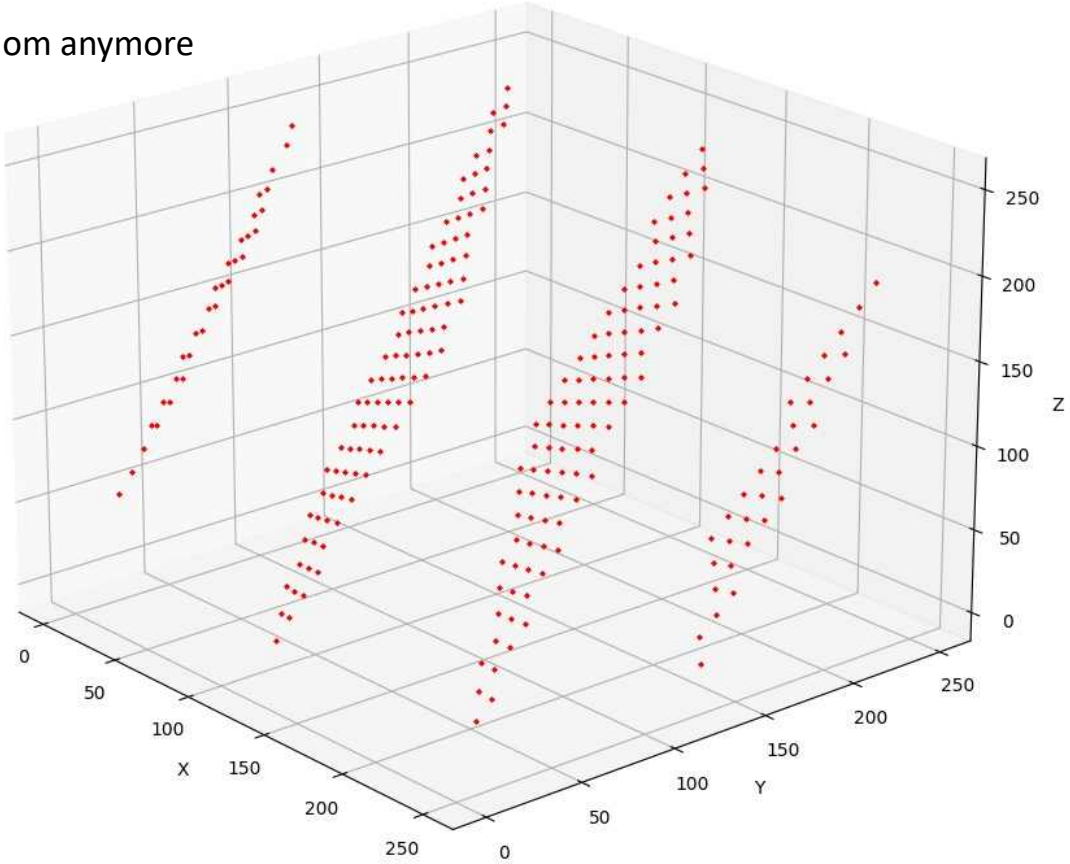
Can you predict this curve?



$f(x) = 77x$ modulo 251



Drawing x, y, z: not so random anymore



Go through https://webhome.phy.duke.edu/~rgb/General/dieharder.php

Abstract

Dieharder: A Random Number Test Suite

Version 3.31.1

Robert G. Brown (rgb)

Dirk Eddelbuettel

David Bauer

Welcome to the dieharder distribution website.

Version 3.29.4beta is the current snapshot. Some of the documentation below may not quite be caught up to it, but it should be close.

Dieharder is a *random number generator (rng) testing suite*. It is intended to test *generators*, not *files of possibly random numbers* as the latter is a fabulous view of what it means to be random. Is the number '7' random? It is generated by a random process, it might be. If it is made up to serve the purpose of some arguments (like this one) it is not. Perfect random number generators produce 'unlikely' sequences of random numbers — at exactly the right average rate. Testing a rng is therefore quite subtle.

dieharder is a tool designed to permit one to push a weak generator to interesting failure (to the e.g. 0.000001% level, not leave one in the 'limbo' of 1% or 5% mere failure. It also contains many tests and is extensible so that eventually it will contain many more tests than it already does.

If you are using dieharder for testing rngs either in one of its prebuilt versions (rpm or apt) or built from source (which gives you the ability to use add more tests or integrate your rng directly with dieharder for ease of use) you may want to join either or both of the dieharder-announce or the dieharder-devel mailing lists here. The former should be very low traffic — literally announcing when a completed make it through development to where I'm proud of it. The latter will be a bit more active, and is a good place to post bug reports, patches, suggestions, fixes, complaints and generally participate in the development process.

About Dieharder

At the suggestion of Linus Torvalds on the Gnu Scientific Library (GSL) list this GPL'd suite of random number tests will be named "Dieharder". Using a movie/sequel pun for the name is a double tribute to George Marsaglia, whose "Diehard battery of tests" of random number generators has enjoyed years of enduring usefulness as a test suite.

The dieharder suite is more than just the diehard tests relearned up and given a pretty GPL'd source face to native C. Tests from the Statistical Test Suite (STS) developed by the National Institute for Standards and Technology (NIST) are being incorporated, as are new tests developed by rgb. Where possible or appropriate, *all* tests that can be parameterized ("cranked up") to where failure, at least, if unambiguous are so parameterized and controllable from the command line.

A further design goal is to provide some indication of why a generator fails a test, where such information can be extracted during the test process and placed in usable form. For example, the KS distribution tests should (eventually) be able to display the actual histogram for the different bit samplers.

Dieharder is by design generative. It is intended to be the "Swiss army knife of random number test suites" (or if you prefer, "the last suite you'll ever want" for testing random numbers.

Random Numbers

https://en.wikipedia.org/wiki/Pseudorandom_number_generator

https://www.youtube.com/watch?v=_tN2ev3hO14

https://en.wikipedia.org/wiki/Linear_congruential_generator

https://stackoverflow.com/questions/2130621/how-to-test-a-random-generator

https://webhome.phy.duke.edu/~rgb/General/dieharder.php

https://csrc.nist.gov/projects/random-bit-generation/documentation-and-software/guide-to-the-statistical-tests

http://simul.iro.umontreal.ca/testu01/tu01.html

Random numbers in Rust

```rust
// cargo add rand
use rand::distributions::{Distribution, Uniform};

fn main() {
    let mut rng      = rand::thread_rng();
    let distribution = Uniform::from(0.0f32 .. 1.0f32);

    for _ in 0..10 {
        let r = distribution.sample(&mut rng);
        println!("{r}");
    }
    let distribution = Uniform::from(1 .. 100);
    for _ in 0..10 {
        println!("{}", distribution.sample(&mut rng));
    }
}
```

```
0.6430509
0.7014921
0.8340076
0.3438382
0.40795302
0.34134233
0.7871982
0.04053259
0.55015457
0.95575976
48
81
85
18
79
32
32
64
60
77
```

https://crates.io/crates/rand

https://rust-random.github.io/book

A Rust library for random number generation, featuring:

- Easy random value generation and usage via the `Rng`, `RngCore` and `CryptoRng` traits
- Secure seeding via the `getrandom` crate and fast, convenient generation via `thread_rng`
- A modular design built over `rand_core` (see the book)
- Fast implementations of the best-in-class cryptographic and non-cryptographic generators
- A flexible `distributions` module
- Samplers for a large number of random number distributions via our own `rand_distr` and via the `statrs`
- Portably reproducible output
- `#[no_std]` compatibility (partial)
- *Many* performance optimisations

It's also worth pointing out what `rand` is *not*:

- Small. Most low-level crates are small, but the higher-level `rand` and `rand_distr` each contain a lot of functionality.
- Simple (implementation). We have a strong focus on correctness, speed and flexibility, but not simplicity. If you prefer a small-and-simple library, there are alternatives including `fastrand` and `oorandom`.
- Slow. We take performance seriously, with considerations also for set-up time of new distributions, commonly-used parameters, and parameters of the current sample.

Reliability is about consistency

Ask:

"Would I get the same result again?"

Validity is about accuracy

Ask:

"Am I measuring what I think I am measuring?"

How a computer works

What would we need for a minimal programming language

1. Ability to store our program, e.g., line by line
2. Do computations on numbers, like, `Line 7: 4*5+7/8`
3. Store those numbers and retrieve them:
   ```
   Line 98: storage[123] := 17+4
   Line 99: storage[128] := storage[123] - 5
   ```
4. Depending whether a number is zero jump somewhere in the program, e.g.
   ```
   Line 107: if storage[55] + 8 = 0 goto line 70
   ```

Actually, that is already more than what we need.

- ▷ Now we can build, the usual control instructions, like, if, while, for, easily from this.
- ▷ How do we do subroutines?
- ▷ What is happening inside the computer?

Tao of programming

The Tao gave birth to machine language.

Machine language gave birth to the assembler.

The assembler gave birth to the compiler.

Now there are ten thousand languages.

Each language has its purpose, however humble.

Each language expresses the Yin and Yang of software.

Each language has its place within the Tao.

But do not program in COBOL if you can avoid it.

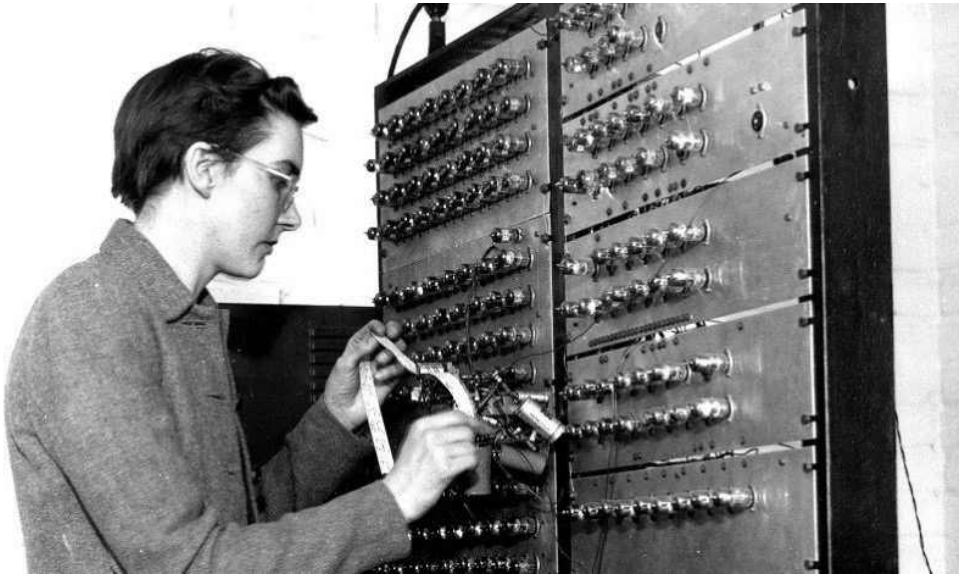
https://webhome.phy.duke.edu/~rgb/General/tao/tao.html

Kathleen Booth (*9 Jul 1922, †29 Sep 2022)

Kathleen Hylda Valerie Booth (née Britten) was a British computer scientist and mathematician who wrote the first assembly language and designed the assembler and autocode for the first computer systems at Birkbeck College, University of London. She helped design three different machines including the ARC (Automatic Relay Calculator), SEC (Simple Electronic Computer), and APE(X)C.

https://en.wikipedia.org/wiki/Kathleen_Booth

https://www.golem.de/news/nachruf-assembl
169273.html

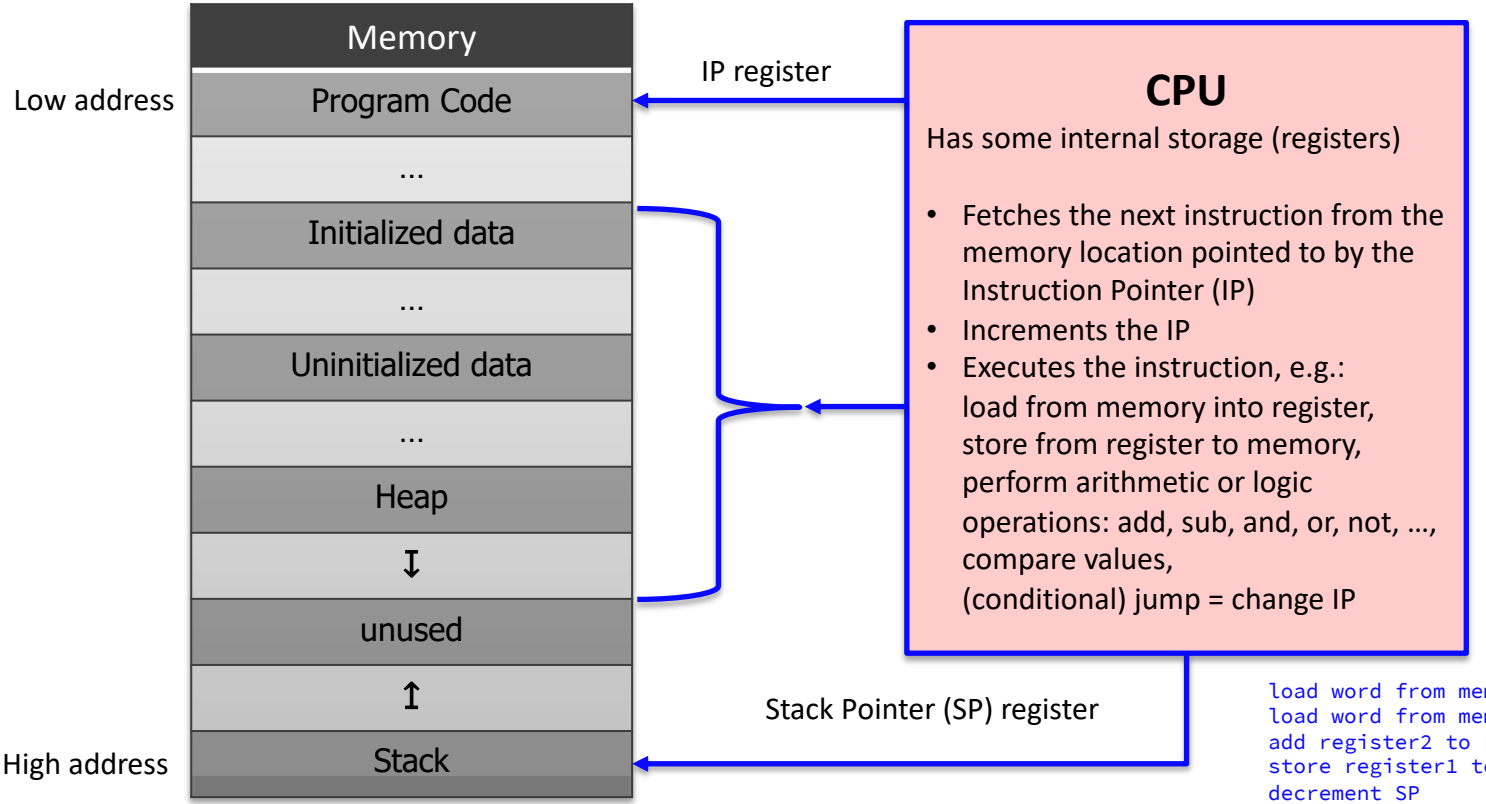


The von Neumann architecture — also known as the von Neumann model or Princeton architecture — is a computer architecture based on a 1945 description by John von Neumann, and by others, in the First Draft of a Report on the EDVAC. The document describes a design architecture for an electronic digital computer with these components:

- A processing unit with both an arithmetic logic unit and processor registers
- A control unit that includes an instruction register and a program counter
- **Memory that stores data and instructions**
- External mass storage
- Input and output mechanisms

The term “von Neumann architecture” has evolved to refer to any stored-program computer in which an instruction fetch and a data operation cannot occur at the same time (since they share a common bus). This is referred to as the von Neumann bottleneck, which often limits the performance of the corresponding system.

Computer (memory/registers in *Words*, e.g., 16, 32, 64 bit)




```
load [100], R1
load [101], R2
add R2, R1
push R1
dec R2
test R2
jnz 53
```

```
load word from mem address 100 to register1
load word from mem address 101 to register2
add register2 to register1
store register1 to mem address in SP, and
decrement SP
decrement R2
compare R2 with 0
if not 0 jump to address 53 (set IP = 53)
```

Question: Where is the time spent, when a program runs?

```
fib_iterative3(unsigned int):
   lea ecx, [rdi-1]
   test edi, edi                    if n == 0 goto .L4
   je .L4
   mov edx, 1                       b = 1
   xor eax, eax                     a = 0
.L3:
   xadd eax, edx                    t = a + b, a = b, b = t
   sub ecx, 1                       n = n - 1
   jnb .L3                          if n != 0 goto .L3
   ret
.L4:
   xor eax, eax                     a = 0
   ret
```

What makes computer different

	e.g.	
CPU Architecture	x86, x86-64, ARM, Power, Risc-V	
CPU subtype	x86, haswell, skylake, broadwell, zen3, armv7, armv8, power9, RV32E, R128I	
Operating system	Linux, Windows, MaxOS, AIX, Solaris	
Binary/object file format	ELF, COFF, MACH-O,	https://en.wikipedia.org/wiki/Executable_and_Linkable_Format https://en.wikipedia.org/wiki/Mach-O

file(1) command

- `ELF 64-bit LSB pie executable, x86-64, version 1 (GNU/Linux), dynamically linked, for GNU/Linux 3.2.0, not stripped`
- `ELF 64-bit LSB executable, x86-64, version 1 (GNU/Linux), statically linked, for GNU/Linux 3.2.0, stripped`
- `ELF 32-bit LSB executable, ARM, EABI5 version 1 (SYSV), dynamically linked, for GNU/Linux 3.2.0, with debug_info, not stripped`
- `Mach-O 64-bit x86_64 executable, flags:<NOUNDEFS|DYLDLINK|TWOLEVEL|WEAK_DEFINES|BINDS_TO_WEAK|PIE>`
- `PE32+ executable (console) x86-64, for MS Windows`

Note: Whenever something is referred to as **name(X)**, this means it is a command or function and you can get the description by saying:

```
$ man X name    # e.g., man 1 file  The X is optional, but there may be several
```

Student presentation

```
v,i,j,k,l,s,a[99];
main(){for(scanf("%d",&s);*a-s;v=a[j*=v]-
a[i],k=i<s,j+=(v=j<s&&(!k&&!!printf(2+"\n\n%c"-(!l<<!j),"
#Q"[l^v?(l^j)&1:2])&&++l||a[i]<s&&v&&v-i+j&&v+i-
j))&&!(l%=s),v||(i==j?a[i+=k]=0:++a[i])>=s*k&&++a[--i]);}
```

Not needed ☺

What might it possibly do?

Solution to some programming exercise in COMA (python):

```
L = [4,5,3,2,1,3,6,4,7]
print(1 + len(max("".join(list(map(lambda a:"RGH"[(a[1] > a[0]) -
(a[1] < a[0]) + 1], zip(L[:-1],L[1:])))).replace("RH", "R H").split(),
key=len)))
```

ChatGPT5

Short answer: it prints **5**.

What it's doing:

Compares each adjacent pair in L and maps the step to:

"H" if next > current (up),

"G" if equal (flat),

"R" if next < current (down).

It joins those letters into a string, then inserts a space at every **valley** pattern "RH" (down then up), and splits on those spaces.

The longest chunk (by length) is a run that never has a valley—i.e., a **bitonic** run (increasing then decreasing, with flats allowed).

It adds 1 to convert "number of steps" to "number of elements" and prints that.

For your L = [4,5,3,2,1,3,6,4,7], the direction string is HRRRHHRH. Splitting on "RH" gives chunks ["HRRR", "HHR", "H"]; the longest has length 4, so it prints 4 + 1 = 5.

So: it computes the **length of the longest contiguous bitonic subarray** (allowing equal adjacents) and prints it.

A **bitonic** sequence (or subarray) is one that **first increases** and then **decreases** — it has **at most one "peak."**

Formally:

It's a sequence of numbers that can be divided into two parts:

The first part is **non-decreasing** (each element ≥ previous).

The second part is **non-increasing** (each element ≤ previous).

Examples:

[1, 3, 7, 5, 2] → increases (1→3→7), then decreases (7→5→2). ✅ bitonic

[2, 4, 6, 8, 9] → only increasing (no decrease yet). ✅ bitonic (purely increasing is allowed)

[9, 7, 5, 3] → only decreasing. ✅ bitonic (purely decreasing is allowed)

[1, 4, 2, 5, 3] → goes up, down, then up again. ❌ **not** bitonic (more than one peak)

So in your program:

It's finding the **longest contiguous bitonic subarray** in L —

the longest stretch that rises (maybe stays flat) and then falls (maybe stays flat) once.

Please start each file with a comment

```
// Sabine Mustermann
```

Rust style guide: https://doc.rust-lang.org/nightly/style-guide/index.html

The Rust reference: https://doc.rust-lang.org/reference

The Rust programming language (TRPL): https://doc.rust-lang.org/book/title-page.html

The rustc book: https://doc.rust-lang.org/stable/rustc/index.html

The Cargo book: https://doc.rust-lang.org/stable/cargo/index.html

Go through https://en.wikipedia.org/wiki/Rust_(programming_language)

Show fibonacci.rs

Try not to be too clever (From Kernighan, Plauger, The Elements of P. Style)

```rust
fn main() {
    const N : usize = 5;  // Matrix size

    // Define an NxN matrix using arrays
    let mut v: [[f64; N]; N] = [[0.0; N]; N];

    for i in 1..=N {
        for j in 1..=N {
            v[i-1][j-1] = ((i/j) * (j/i)) as f64;
        }
    }

    // Iterate through the matrix
    // and print all elements
    for row in &v {
        for element in row {
            print!("{} ", element);
        }
        println!();
    }
}
```

```rust
// Better way to achieve the same

for i in 0..N {
    for j in 0..N {
        v[i][j] = if i == j { 1.0 } else { 0.0 };
    }
}


for i in 0..N {
    for j in 0..N {
        v[i][j] = 0.0;
    }
    v[i][i] = 1.0;
}

// Notes:
// - Not the preferred way to iterate through arrays
// - in the example as it is here, there is no point
//   in setting off diagonal values to zero, this was
//   done when the matrix was defined.
```

Getting command line arguments

```rust
use std::env;

fn main() {
    let args: Vec<String> = env::args().collect();

    for s in &args {
        println!("{}", *s);
    }
    let argc = args.len();

    if argc < 3 {
        eprintln!("usage: {} number filename\n", &args[0]);
        std::process::exit(1);
    }
    assert!(argc > 2);

    let num : i64 = args[1].parse().unwrap();
    let str       = args[2].clone();

    println!("num = {num}");
    println!("str = {str}");
}
```

```
if (country == SING) || (country == BRNI) ||
   (country == POL) || (country = ITALY) {
//
// If the country is Singapore, Brunei or Poland
// then the current time is the answer time
// rather than the off hook time.
// Reset answer time and set day of week.
//
…
```

NEEDS UPDATE. Country = ITALY doesn‘t work in RUST.

See https://www.youtube.com/watch?v=Bf7vDBBOBUA&t=332s

Don't Write Comments

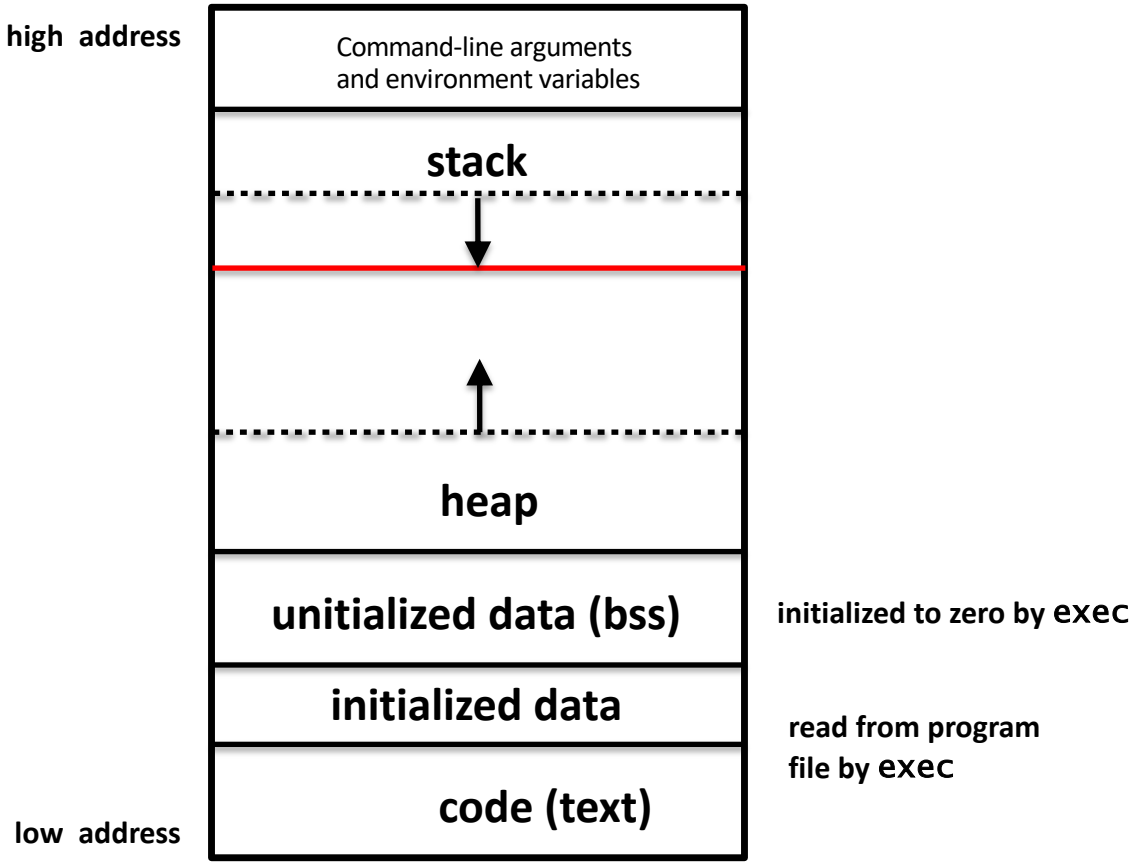
```
let the_element_index = 0;

while the_element_index < number_of_elements {
    element_array[the_element_index] = the_element_index;
    the_element_index = the_element_index + 1;
}
```

Better:

```
for i in 0..nelems {
   elem[i] = i;
}
```

Memory layout of C/C++ programs



From http://www.geeksforgeeks.org/memory-layout-of-c-program/

Code Segment

A code segment , also known as a text segment, is one of the sections of a program in an object file or in memory, which contains executable instructions.

As a memory region, a text segment may be placed below the heap or stack in order to prevent heaps and stack overflows from overwriting it.

Usually, the text segment is sharable so that only a single copy needs to be in memory for frequently executed programs, such as text editors, the C compiler, the shells, and so on. Also, the text segment is often read-only, to prevent a program from accidentally modifying its instructions.

The stack area traditionally adjoined the heap area and grew the opposite direction; when the stack pointer met the heap pointer, free memory was exhausted. (With modern large address spaces and virtual memory techniques they may be placed almost anywhere, but they still typically grow opposite directions.)

The stack area contains the program stack, a LIFO structure, typically located in the higher parts of memory. On the standard PC x86 computer architecture it grows toward address zero; on some other architectures it grows the opposite direction. A “stack pointer” register tracks the top of the stack; it is adjusted each time a value is “pushed” onto the stack. The set of values pushed for one function call is termed a “stack frame”; A stack frame consists at minimum of a return address.

Stack, where automatic variables are stored, along with information that is saved each time a function is called. Each time a function is called, the address of where to return to and certain information about the caller’s environment, such as some of the machine registers, are saved on the stack. The newly called function then allocates room on the stack for its automatic and temporary variables. This is how recursive functions in C can work. **Each time a recursive function calls itself, a new stack frame is used, so one set of variables doesn’t interfere with the variables from another instance of the function.**

Stack overflow

```rust
fn f(count: usize) {
    let _large_local_variable: [u8; 1_000_000] = [0; 1_000_000];
    println!("{count} ");
    if count < 100 { // silence warning
        f(count + 1);
    }
}

fn main() {
    recurse(1);
}
```

```
1  2  3  4  5  6  7  8
thread 'main' has overflowed its stack
fatal runtime error: stack overflow
Aborted (core dumped)
```

Heap

Heap is the segment where dynamic memory allocation usually takes place.

The heap area begins at the end of the BSS segment and grows to larger addresses from there.

The Heap area is managed by malloc(), realloc(), and free(),
which may use the brk() and sbrk() system calls to adjust its size.

(note that the use of brk/sbrk and a single “heap area” is not required to fulfill the contract of malloc/realloc/free; they may also be implemented using mmap to reserve potentially non-contiguous regions of virtual memory into the process’ virtual address space).

The Heap area is shared by all shared libraries and dynamically loaded modules in a process.

Fragen

有問題嗎

คำถาม

質問

Вопросы

Questions

Câu hỏi